

Odoo Invoice Agent — Ecotel Italia

Agent Python per l'automazione della registrazione delle fatture passive (e-fatture XML FatturaPA) in **Odoo 14**. Analizza le fatture elettroniche in ingresso, le classifica, e per alcuni fornitori predisposti crea **bozze** `account.move` **pre-compile** nel sistema.

Questo file serve a dare a chi arriva sul progetto (umano o AI agent) il contesto necessario per lavorarci senza dover ricostruire tutto da zero.

1. Scopo e stato attuale

Obiettivo: ridurre il lavoro manuale di registrazione delle fatture fornitore ricorrenti.

Pipeline dell'agent:

1. Legge le e-fatture in ingresso da Odoo (`fatturapa.attachment.in`) con `registered=False`, `is_self_invoice=False`, filtrato per company)
2. Estrae dati XML (FatturaPA): partner, importi, righe, tipo documento (TD01, TD04, ecc.)
3. Classifica ogni fattura in una categoria (MAPPATURA_FORNITORE_FISSO, AUTO_VALIDABILE, DA_VERIFICARE, NO_ODA, ...)
4. Per i fornitori fissi, crea bozze `account.move` in Odoo collegate a un OdA ricorrente ("contenitore annuale")
5. Produce report (Excel + dashboard webapp)

Fornitori in produzione con bozze automatiche (mappatura in `config/rules.py`):

- **Trenitalia S.p.A.** (PIVA IT05403151003) → OdA `P03524`, conto `420173` (spese di viaggio)
- **Italo NTV** (PIVA IT09247981005) → OdA `P04279`, stesso conto

Tipi documento supportati: TD01 (fattura) e TD04 (nota di credito). Altri tipi (TD05, TD06...) attualmente rifiutati.

2. Architettura dei file

```
odoo_invoice_agent/  
├─ config/  
│   ├─ credentials.env           # URL Odoo, user, password (NON committare)  
│   ├─ credentials.env.template  
│   └─ rules.py                 # MAPPATURA_FORNITORI_FISSI, flag globali  
├─ core/  
│   └─ odoo_client.py           # XML-RPC client READ-ONLY per Odoo
```

```

|   ├── odoo_writer.py           # Scritture Odoo (create bozza, rollback)
|   ├── fatturapa_parser.py      # Parser XML FatturaPA → dataclass
|   ├── classifier.py           # Logica di classificazione fatture
|   └── (altri moduli: analyzer, matcher, ecc.)
└── webapp/
    ├── app.py                  # Dashboard Flask
    ├── templates/              # Jinja2
    └── dashboard.db             # SQLite audit trail (NON committare)
└── reports/                    # Generatori Excel
└── output/                     # Report giornalieri (NON committare)
└── logs/                       # Log applicativi (NON committare)
└── run_agent.py                # Entry point CLI
└── requirements.txt            # python-dotenv, openpyxl, flask
└── CLAUDE.md                   # Questo file

```

Separazione writer vs client

Convenzione importante: `odoo_client.py` è **read-only**, `odoo_writer.py` è l'unico modulo che scrive su Odoo. Questo serve a localizzare le operazioni pericolose in un unico file auditabile.

DB dashboard.db

SQLite con due tabelle principali:

- `analyses`: una riga per ogni fattura analizzata in una run (inclusi `xml_data` e `raw_xml`)
- `odoo_writes`: audit trail delle bozze create dall'agent (`move_id`, `po_line_id`, `old_price_unit`, `old_name`, `old_date_planned`, `timestamp`). Serve per rollback e per polling sullo stato delle bozze (detect cancellazione manuale).

3. Deploy e ambienti

PC locale dell'utente (sviluppo/test):

```
C:\Users\lranalletta\Documents\AGENT FATTURAZIONE PASSIVA\odoo_invoice_agent
```

Python 3.12, webapp su `(127.0.0.1:5000)` (default Flask).

Server aziendale Windows (produzione):

```
C:\odoo_apps\invoice_agent
```

Python 3.9+, webapp modificata con `(host='0.0.0.0', port=80)` per essere raggiungibile dalla rete aziendale. Accesso via RDP.

⚠ **Divergenza server vs locale**

Il file `webapp/app.py` è **diverso tra locale e server**. Sul server l'ultima riga è:

```
python  
  
app.run(debug=False, host='0.0.0.0', port=80, threaded=True)
```

In locale resta `127.0.0.1:5000`. **Se si aggiorna `app.py`, fare merge manuale** per non sovrascrivere la modifica server. Idealmente andrebbe parametrizzato via env var (`WEBAPP_HOST`, `WEBAPP_PORT`) — task aperto.

Workflow di deploy

Attualmente manuale: modifica locale → test → copia via RDP dei file modificati sul server (non l'intero zip, per preservare le modifiche puntuali come `app.py`). Prossimo step: valutare Git.

4. Convenzioni critiche

Mappatura fornitori fissi (`config/rules.py`)

Struttura `MAPPATURA_FORNITORI_FISSI[partita_iva]`:

```
python  
  
{  
    'nome': str,  
    'oda_fisso': str,          # es. 'P03524'  
    'partner_id': int,        # id Odoo del res.partner  
    'conto_contabile_id': int, # id Odoo del account.account  
    'taxes_id': [int],        # es. [12] per IVA 10%  
    'journal_id': int,  
    'company_id': int,  
    'libere_criterio': str,    # 'standard_qty_inv_rec' (default) o 'price_zero_  
    'description_strategy': str, # 'trenitalia_titoli' o 'pass_through'  
    'auto_write_enabled': bool,  
}
```

Criteri "riga libera" (quale riga OdA consumare per appenderci la fattura):

- `standard_qty_inv_rec` (default): `qty_invoiced=0 AND qty_received=0 AND product_qty>=1`. Ignora prezzo e descrizione. Adatto per OdA-ledger ricorrenti.
- `price_zero_only` (legacy): solo `price_unit=0 AND qty_invoiced=0`.

Strategie descrizione riga:

- `trenitalia_titoli`: estrae codici biglietto da `<AltriDatiGestionali><TipoDato>Tit.n.X</TipoDato>...` e costruisce `"TRATTA (Tit. CODICI, DATA)"`
- `pass_through`: usa la descrizione XML così com'è, aggiungendo solo la data. Adatto per Italo (che già ha codice+tratta+viaggiatore nella descrizione).

Note di credito (TD04)

Gestione particolare:

- Nel move: `move_type='in_refund'`, importo **positivo** (Odoo gestisce il segno contabile internamente)
- Nella riga OdA: `price_unit` **negativo** (per compensare la riga gemella positiva della fattura originale)
- Descrizione con prefisso `"NC - "` + eventuale `"rif.ft <numero fattura originale>"` estratto da `<AltriDatiGestionali><TipoDato>FATTURA</TipoDato>`

Punto aperto (da confermare con contabilità Ecotel): il move_line dell'agent ha attualmente `quantity=+1, price_unit=-X`. Il manuale della contabilità usa `quantity=-1, price_unit=-X` (subtotal `+X`, move totale positivo). Il test del post in Odoo dirà se serve il fix. Se serve, è una riga da cambiare in `odoo_writer.py` (`'quantity': -1 if is_nota_credito else 1`).

Data competenza IVA

Convenzione italiana: `date` e `l10n_it_vat_settlement_date` del move = **fine mese** della data fattura. Helper `_end_of_month()` in `odoo_writer.py`. Vale anche per le NC.

Campi obbligatori nel move_line

Via XML-RPC gli onchange di Odoo non vengono triggerati, quindi **tutti i campi che la UI autopopolerebbe vanno passati esplicitamente**. In particolare:

- `product_id`: dalla riga OdA consumata (`po_line['product_id'][0]`)
- `product_uom_id`: dalla riga OdA (`po_line['product_uom'][0]`) — nota il nome diverso: `purchase.order.line.product_uom` vs `account.move.line.product_uom_id`

Dimenticare questi campi lascia la colonna "Prodotto/Categoria" vuota nella bozza (bug visto e corretto).

Rollback e idempotenza

Ogni `create_bozza_fornitore_fisso` salva in DB i valori pre-modifica della riga OdA (`old_price_unit`, `old_name`, `old_date_planned`). Il rollback:

1. Verifica che il move sia ancora in `draft` (rifiuta posted per sicurezza)
2. `unlink` del move

3. Ripristino riga OdA ai vecchi valori + `qty_received=0, qty_received_manual=0`

4. De-registrazione dell'attachment fatturapa (lo riporta in "da registrare")

Esiste anche `restore_po_line` per il caso in cui la bozza venga cancellata manualmente in Odoo (il polling lo rileva).

Safety: DRY_RUN

`ODOO_WRITE_DRY_RUN` in `config/rules.py`. Se True, tutto viene loggato ma nulla scritto. **Da usare come interruttore di emergenza.**

5. XML-RPC gotchas di Odoo 14

- `write` si chiama con due argomenti posizionali: `_call('purchase.order.line', 'write', [po_line_id], {vals})`. **NON** wrappare `[po_line_id]` in un'altra lista o si ottiene `unhashable type: 'list'`.
 - `create` può ritornare un int o una lista di int a seconda del modello. Normalizzare sempre: `if isinstance(move_id, list): move_id = move_id[0]`.
 - I campi Many2one in output sono tuple `[id, display_name]`. Accedere sempre con `[0]` dopo check `get()`.
 - Onchange **NON** vengono eseguiti via XML-RPC. Tutti i campi auto-popolati via UI vanno passati esplicitamente.
-

6. Come lanciare

Agent CLI (analisi + report + eventuali bozze)

```
powershell
cd "C:\odoo_apps\invoice_agent"
python run_agent.py --from 2026-04-20 --to 2026-04-22
```

Webapp dashboard

```
powershell
cd "C:\odoo_apps\invoice_agent"
python webapp\app.py
```

Locale: `http://localhost:5000`

Server: `http://<ip-server>` (porta 80)

Test di sintassi rapido dopo modifiche

```
powershell
```

```
python -c "import ast; ast.parse(open('core/odoo_writer.py').read()); print('OK')"
```

7. Credenziali e sicurezza

- `config/credentials.env` contiene `ODOO_URL`, `ODOO_DB`, `ODOO_USERNAME`, `ODOO_PASSWORD`. **Mai committare** (è nel `.gitignore` se Git verrà attivato).
- La webapp sul server è esposta sulla porta 80 senza autenticazione. Chiunque in rete aziendale può creare bozze. **Aggiungere login** è un task aperto prioritario una volta consolidato il flusso.
- Utente Odoo attualmente usato per l'agent: `lranalletta@ecotelitalia.it` (utente reale, non di servizio). **Da migrare** a service account dedicato (es. `agent_fatture@ecotelitalia.it`) per audit e sicurezza.

8. TODO / backlog aperti

1. **Feedback contabilità sul segno quantity NC** + test post della NC agent in Odoo
2. **Parametrizzare host/port webapp** via env var per eliminare divergenza locale/server
3. **Login sulla webapp** (Flask-Login minimo)
4. **Servizio Windows** per avviare webapp automaticamente al boot del server
5. **Schedulazione Task Scheduler** per run notturno di `run_agent.py`
6. **Email contabili** (parametri SMTP da chiedere a IT)
7. **Fase 2 — matching commesse S#####** (sale.order per imputazione costi a progetti)
8. **Estensione a fornitori non-OdA** (utenze, leasing, ecc.)
9. **Utente Odoo dedicato** (service account)
10. **Git + repo aziendale** per eliminare copy-paste RDP

9. Stile di collaborazione preferito dall'utente

- Comunicazione in **italiano**
- **Ragionare prima di agire**: spiegare il "perché" delle scelte tecniche, mostrare trade-off, chiedere conferma prima di modifiche invasive

- Modifiche **chirurgiche**: preferire `str_replace` mirati sui singoli file invece di rigenerare intero zip, per non sovrascrivere modifiche server (es. `app.py`)
- Validare ipotesi con **script diagnostici** prima di scrivere codice di produzione
- Evitare overengineering: la soluzione minima che funziona è preferibile